

Module 1: Automata, Formal Languages & Modern Systems

CS 305: Theory of Computation (Weeks 1–5)

Department of Computer Science

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate

Module 1 Roadmap: Theory $\leftrightarrow$ Cutting-Edge Systems

Theoretical Foundations

- **Wk 1:** Formal Languages & Chomsky Hierarchy
- **Wk 2:** Deterministic Finite Automata (DFAs)
- **Wk 3:** NFAs & Powerset Construction Algorithm
- **Wk 4:** Regular Expressions & GNFA State Elimination
- **Wk 5:** Non-regular Languages & Pumping Lemma

Research Literature & System Spotlights

- **Wk 1: ICLR 2023:** Neural Networks & Chomsky Hierarchy
- **Wk 2: ACM SIGCOMM:** 100 Gbps Packet Inspection
- **Wk 3: CACM:** Linear vs. Exponential Regex Engines
- **Wk 4: USENIX Security 2018:** ReDoS Vulnerabilities
- **Wk 5:** Bounded Memory in Edge Hardware & AI Models

Alphabets, Strings, and Languages (Sipser Ch. 0)

- **Alphabet (Σ):** A non-empty, finite set of symbols (e.g., $\Sigma = \{0, 1\}$, $\Sigma = \{a, b, \dots, z\}$).
- **String (w):** A finite sequence of symbols chosen from Σ . Length $|w|$ is the number of symbols. The empty string is denoted ε ($|\varepsilon| = 0$).
- **Language (L):** A set of strings over an alphabet ($L \subseteq \Sigma^*$).
- **Kleene Star (Σ^*):** The set of all possible finite strings over Σ , including ε .

Language Example

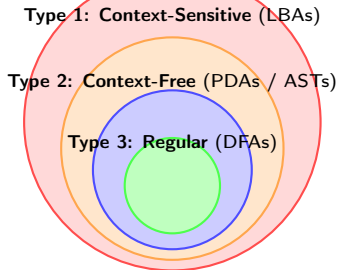
Let $\Sigma = \{0, 1\}$. Then $L = \{w \in \Sigma^* \mid w \text{ has an even number of 0's}\}$ is a formal language over Σ .

Week 1 Research Spotlight: Neural Networks & The Chomsky Hierarchy

Seminal Reading: ICLR 2023 (Spotlight)

Delétang et al. (Google DeepMind), “Neural Networks and the Chomsky Hierarchy,” International Conference on Learning Representations (ICLR 2023).

Type 0: Rec. Enumerable (Turing / CoT LLMs)



Key Empirical Findings

- Standard Transformers without memory fail to generalize on non-regular languages ($a^n b^n c^n$).
- Memory-augmented neural models track the Chomsky hierarchy with mathematical precision!

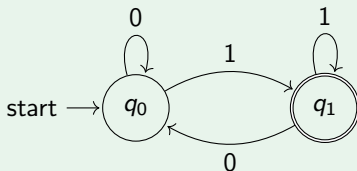
Week 2: Deterministic Finite Automata (DFA)

Formal Definition (Sipser Def 1.5)

A **Deterministic Finite Automaton (DFA)** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

- 1 Q is a finite set of **states**.
- 2 Σ is a finite set of symbols (**alphabet**).
- 3 $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**.
- 4 $q_0 \in Q$ is the **start state**.
- 5 $F \subseteq Q$ is the set of **accept (final) states**.

DFA State Diagram: Strings ending with '1'



Week 2 Research Spotlight: High-Speed Deep Packet Inspection

Seminal Reading: ACM SIGCOMM

Kumar et al. (Univ. of Wisconsin / Cisco), *"Algorithms to Accelerate Multiple Regular Expression Matching for Deep Packet Inspection,"* **ACM SIGCOMM CCR**.

The Hardware Challenge

- Modern network switches process 100+ Gbps traffic in real time.
- Snort and Zeek firewalls must scan every byte against 10,000+ malware signature patterns.

The DFA Solution: D^2FA

- Hardware-compiled DFAs process each incoming byte in strictly **1 clock cycle** ($\mathcal{O}(1)$ per character).
- Zero packet buffering, zero drops, mathematically optimal throughput!

Week 3: Nondeterministic Finite Automata (NFA)

Formal Definition (Sipser Def 1.16)

An **NFA** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where the transition function allows non-deterministic branching and ε -transitions:

$$\delta : Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q), \quad \text{where } \Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$$

NFA Power & Simplicity

- Allows multiple parallel branches of computation.
- Accepts if *at least one* branch reaches an accept state.
- Exponentially more concise than DFAs for many languages.

Theorem: Equivalence (Sipser Thm 1.39)

Every NFA has an equivalent DFA!

Language Recognizable by NFA $\equiv$ Regular Language

Proof Method: Powerset (Subset) Construction.

Week 3: The Powerset (Subset) Construction

Algorithm: Converting NFA $N = (Q, \Sigma, \delta, q_0, F)$ to DFA $M = (Q', \Sigma, \delta', q'_0, F')$

- 1 Set DFA states to the power set: $Q' = \mathcal{P}(Q)$. (If $|Q| = k$, then $|Q'| \leq 2^k$).
- 2 Start state: $q'_0 = E(\{q_0\})$ (where $E(R)$ is the ε -closure of state set R).
- 3 Transition function: For $R \in Q'$ and $a \in \Sigma$,

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a))$$

- 4 Accept states: $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$.

Inductive Definition of Regular Expressions (Sipser Def 1.52)

R is a regular expression if R is:

- 1 $a \in \Sigma, \varepsilon$, or $\emptyset$ (Atomic base cases).
- 2 $(R_1 \cup R_2)$ (**Union**), $(R_1 \circ R_2)$ (**Concatenation**), or $(R_1)^*$ (**Star**).

Kleene's Theorem

A language L is regular $\iff L$ is described by a regular expression $\iff L$ is recognized by a DFA/NFA.

Seminal Reading: USENIX Security 2018

Staicu & Pradel (TU Darmstadt), "*Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers*," **USENIX Security Symposium 2018**.

The Root Cause: Backtracking Engines

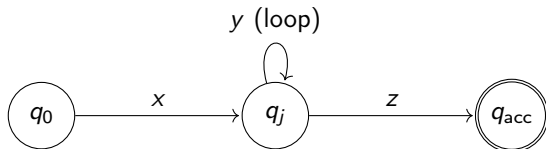
- Python `re`, Node.js, and Java use recursive backtracking for regex matching.
- On pattern $(a^+)+b$ and input a^n , engine evaluates 2^n permutations $\implies$ **Server CPU freeze!**

The Automata Fix (Google RE2 / Rust regex)

- Build an NFA $\rightarrow$ DFA on the fly using Thompson's construction.
- Guarantees $\mathcal{O}(n)$ linear matching time on all inputs!

The Pigeonhole Principle

- A DFA with p states processing a string of length $n \geq p$ must visit at least one state **more than once**.
- Therefore, the computation contains a **loop**.
- The string segment corresponding to the loop can be “pumped” (repeated 0, 2, 3, ... times) and must still be accepted!



String $s = xyz$. For any $i \geq 0$, xy^iz must also be accepted.

Week 5: The Pumping Lemma for Regular Languages (Sipser Thm 1.70)

Formal Theorem Statement

If L is a regular language, then there exists a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be divided into $s = xyz$ satisfying:

- 1 $xy^iz \in L$ for each $i \geq 0$,
- 2 $|y| > 0$,
- 3 $|xy| \leq p$.

Adversarial Game Proof Strategy

To prove $L = \{0^n 1^n \mid n \geq 0\}$ is non-regular:

- 1 **Assume** L is regular (so p exists).
- 2 **Choose** $s = 0^p 1^p \in L$ ($|s| = 2p \geq p$).
- 3 Since $|xy| \leq p$, y consists entirely of 0's ($y = 0^k, k > 0$).
- 4 Pump down with $i = 0$: $xy^0z = 0^{p-k}1^p \notin L$ (fewer 0's than 1's) $\implies$ **Contradiction!**

Module 1 Summary & Looking Ahead to Module 2

Key Takeaways

- **DFA**s, **NFA**s, and **Regular Expressions** are mathematically equivalent representations of **Regular Languages**.
- Regular languages are strictly limited to problems that require **finite, bounded memory**.
- They cannot count or match arbitrary nested structures (0^n1^n or balanced parentheses).

Looking Ahead: Module 2 (Context-Free Languages & Grammars)

How do we parse programming languages, arithmetic expressions, and LLM JSON outputs? We add a **Stack** → **Pushdown Automata & Context-Free Grammars**!